

8.5 Example: “wget” Script Plus “tmpfs” Configuration

In this section, we present a somewhat longer example from our real-life work that combines script programming and system administration.

A tool used on a Linux server is supposed to turn an HTML file into a PDF document. However, the rather old version of the tool was incompatible with modern HTTPS encryption protocols. For various reasons, updating to a new version of the tool was out of the question.

This gave rise to the following idea: it should be possible to use `wget` to download the HTML file, including all additional files referenced there (Cascading Style Sheets [CSS], images, etc.) to a temporary directory. The local files could then be converted to PDF format without network problems.

wget

`wget` is a command commonly used on Linux to download a file via FTP or HTTP. `wget` also has countless special functions that are controlled by options. Among other things, the command can download entire websites. It adapts the links within the HTML pages so that the mirrored website can be viewed locally in the web browser. The trick when using `wget` is to pass the right combination of options.

8.5.1 Developing a “wget” Script

So, the goal was to develop a Bash script that takes all the parameters intended for the tool, creates a temporary directory, downloads the HTML file passed in the first parameter, calls the HTML-PDF converter, and finally cleans everything up. The script was developed in VS Code, with an SSH connection to the server in the editor.

Prompt: I want to write a small Bash script which I pass a URL (first parameter) and some options. The script creates a temporary directory, downloads the URL plus all referenced files (wget relink), calls another command to process these local files, and finally deletes the temporary directory. Can you help? (Short, concise code preferred.)

Certainly! Below is a concise Bash script to achieve what you described:

```
#!/bin/bash
URL=$1
shift    # shift the arguments to remove the URL
# Create a temporary directory
TMPDIR=$(mktemp -d)
# Ensure the temporary directory is deleted on script
exit
trap 'rm -rf "$TMPDIR"' EXIT
# Download the URL and all referenced files into the
# temporary directory
wget --mirror --convert-links --adjust-extension \
      --page-requisites --no-parent -P "$TMPDIR" "$URL"
# Call another command to process these local files
your_command "$TMPDIR" "$@"
```

Explanation

- **Trap:** Ensures that the temporary directory is deleted when the script exits, regardless of success or failure.
- **wget Command:** Uses wget to download the URL and all referenced files into the temporary directory. The options used are:
 - `--mirror`: Enables options suitable for mirroring.
 - `--convert-links`: Converts the links to make them suitable for offline viewing.
 - `--adjust-extension`: Adds the appropriate extensions to files.
 - `--page-requisites`: Downloads all necessary files for displaying the page.
 - `--no-parent`: Prevents wget from following links outside the given URL.
 - `-P "$TMPDIR"`: Specifies the directory to save the files.

- **Processing Command:** Replace `your_command` with the actual command you want to run on the downloaded files. The script passes the temporary directory and any additional options to this command.

The first version of the script already looks quite promising. We've shortened the explanations of the code so that the example remains clear. To try it out, we've adopted the suggestion nearly unchanged and only replaced `your_command` with the absolute path to the HTML PDF converter.

During testing, however, we encountered a few issues, some of which we fixed ourselves and others with AI support. The following prompts document this iterative process. We've refrained from reproducing the rather lengthy answers.

Prompt: I need to pass `$URL` to `your_command`. However, `$URL` needs to be modified to refer to the local start point generated by `wget`.

Prompt: `$URL` can contain all sort of special characters. Can I get `wget` to use `index.html` as a starting point to the local file tree?

ChatGPT doesn't find a suitable option (as far as we know, there is none) and suggests searching for `*.html` files in the temporary directory by using `find` and then using the first file that gets found. However, to us this seems prone to errors:

Prompt: Is it safe to simply take the first result? Perhaps there are other `*.html` files in the directory. Would it make sense to extract the base name from `$URL` and use it as a search pattern with `find`?

ChatGPT now provides code that meets our requirements. However, it turns out that `wget` changes the HTML file too extensively. `wget --mirror` not only downloads the files required to display the HTML page but also processes *all* links contained in the HTML page. As a result, external links in the PDF document that are subsequently forwarded don't work as intended. We try to explain once again what we actually want:

Prompt: I want `wget` to locally download all files needed to correctly display the document (i.e., CSS + images) *and* to change these links within the

document (one document only). But I do *not* want wget to change links to everything else. Links to external files not directly displayed within the document should stay as is. Can you help?

The prompt shows that the wget option `--mirror` originally suggested by ChatGPT was unsuitable. It's pointless to argue about whether we didn't describe the task precisely enough at the beginning or whether the blame lies with ChatGPT. In any case, a relatively long interactive process was required until the script produced in collaboration with ChatGPT worked as intended. The final script looks as follows:

```
#!/bin/bash
URL=$1
shift    # shift the arguments to remove the URL
TMP_DIR=$(mktemp -d /tmp/html2pdf-tmp.XXXXXXXX)
trap 'rm -rf "$TMPDIR"' EXIT
cd "$TMP_DIR"
wget --page-requisites --convert-links --no-parent "$URL"
# if $URL contains a long query string, wget shortens
the file
# name; therefore, find the newest file in the result
list
# with the same base name; this should be reasonably
safe
URL_NO_QUERY=${URL%%\?*}
BASE_NAME=$(basename "$URL_NO_QUERY")
MAIN_FILE=$(find . -type f -name "${BASE_NAME}*" -
print0 | \
                xargs -0 ls -t | head -n 1)
# run html2pdf
/usr/local/bin/html2pdf "$MAIN_FILE" "$@"
```

8.5.2 Linux Server Configuration with “tmpfs”

The script now runs very robustly on a Linux server. To improve efficiency, we've configured the server so that the `/tmp` directory is mapped as a `tmpfs` file system in the working memory. Want to know how to do this? Ask an AI tool!

Prompt: On an Ubuntu server, I want to save the contents of /tmp in memory to make I/O faster. /tmp must not use more than 4 GB RAM. Please show me a systemd compatible setup.

To create a temporary file system in memory for /tmp using tmpfs and limit it to 4 GB of RAM, you can configure a systemd unit for this purpose. Here's how you can set it up:

- First, create a new systemd unit file to mount /tmp as a tmpfs with a 4 GB size limit.

```
sudo nano /etc/systemd/system/tmp.mount
```

[more]

ChatGPT also explains the structure of the tmp.mount file (12 lines of code) and its activation via systemctl daemon-reload and systemctl start tmp.mount. Basically, this is a good solution, but it could be even better. In modern Linux distributions, /lib/systemd/system/tmp.mount already exists. It would be sufficient to make only minimal changes to the existing setup. Not until you put the AI tool on the right track does it describe this procedure:

Prompt: My systemd distribution already contains a preconfigured file tmp.mount. With the exception of the disk size, it's perfect. Can't I simply change the setup to use a maximum of n GB RAM?

Yes, if your system already includes a preconfigured tmp.mount unit file, you can simply modify the existing file to set the maximum size to 4 GB. Here are the steps:

- Open the tmp.mount file for editing:

```
sudo systemctl edit tmp.mount
```

- In the override file, add the following content to set the size limit for the tmpfs mount:

```
[Mount]
Options=mode=1777,size=4G
```

- Reload the `systemd` daemon to apply the changes:

```
sudo systemctl daemon-reload
```

- Restart the `tmp.mount` unit to apply the new size limit:

```
sudo systemctl restart tmp.mount
```

8.5.3 Conclusion

This example was very close to our day-to-day administration work. Did ChatGPT help or not? For the first part of the question, ChatGPT and GitHub Copilot immediately created a basic framework for the script. This part of the task wasn't difficult, but it certainly saved us a few minutes of typing.

The real problem was finding the right mix of options for `wget`. We only use `wget` from time to time and don't know its options by heart. ChatGPT initially led us astray, but ultimately provided the right options. The alternative would have been to look for the options in the manual. This isn't easy because `wget` provides a rather confusing text, the length of which corresponds to about 40 pages of this book. So, it's hard to say whether we would have reached our destination faster this way.

For the second part of the question, the solution proposed by ChatGPT was basically good. The second variant with `systemctl edit` is preferable for modern distributions, however, because it only requires a minimal change to the setup that is already provided. However, you can only access the corresponding instructions if you already know the perfect setup and explicitly point this out to ChatGPT. But, under this assumption, you don't need an AI tool at all.

If we had to grade the whole process, it would likely be a B: the AI tools worked *well*, but not *very well*.